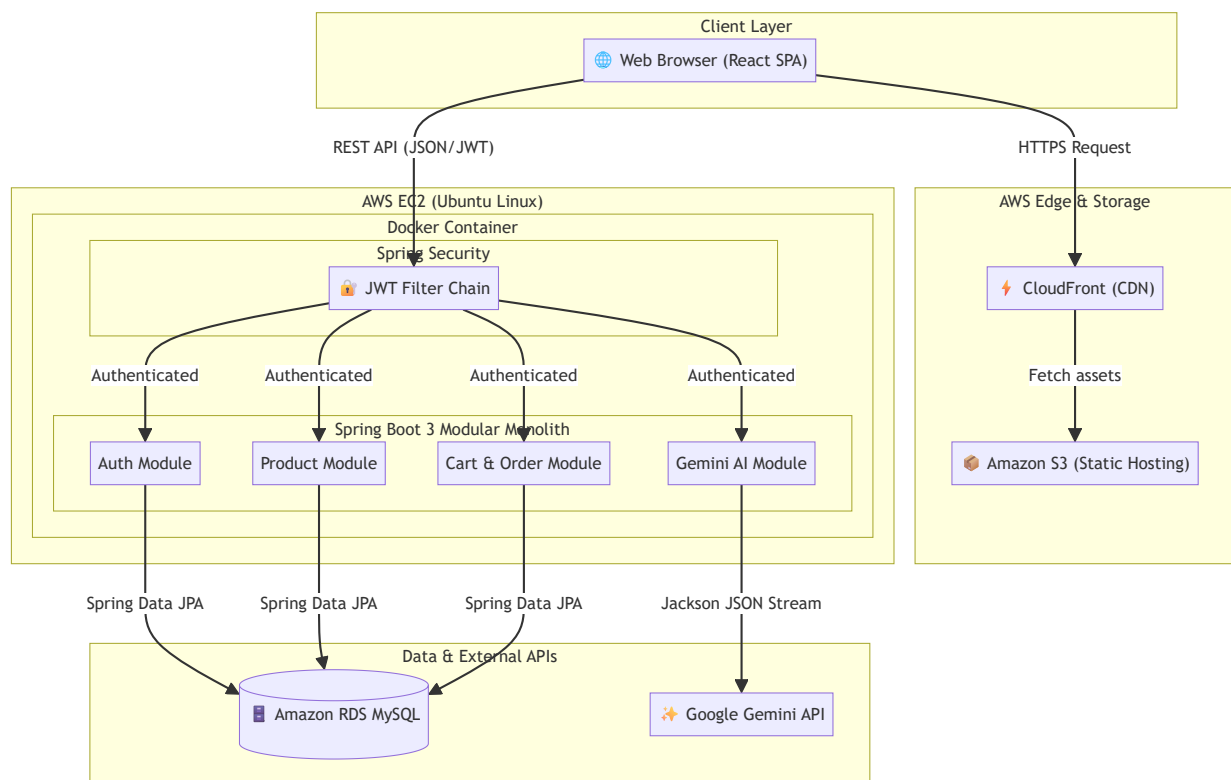


GK ShopEase: Architecture & Execution Workflow

Microservices-Ready Modular Monolith Platform

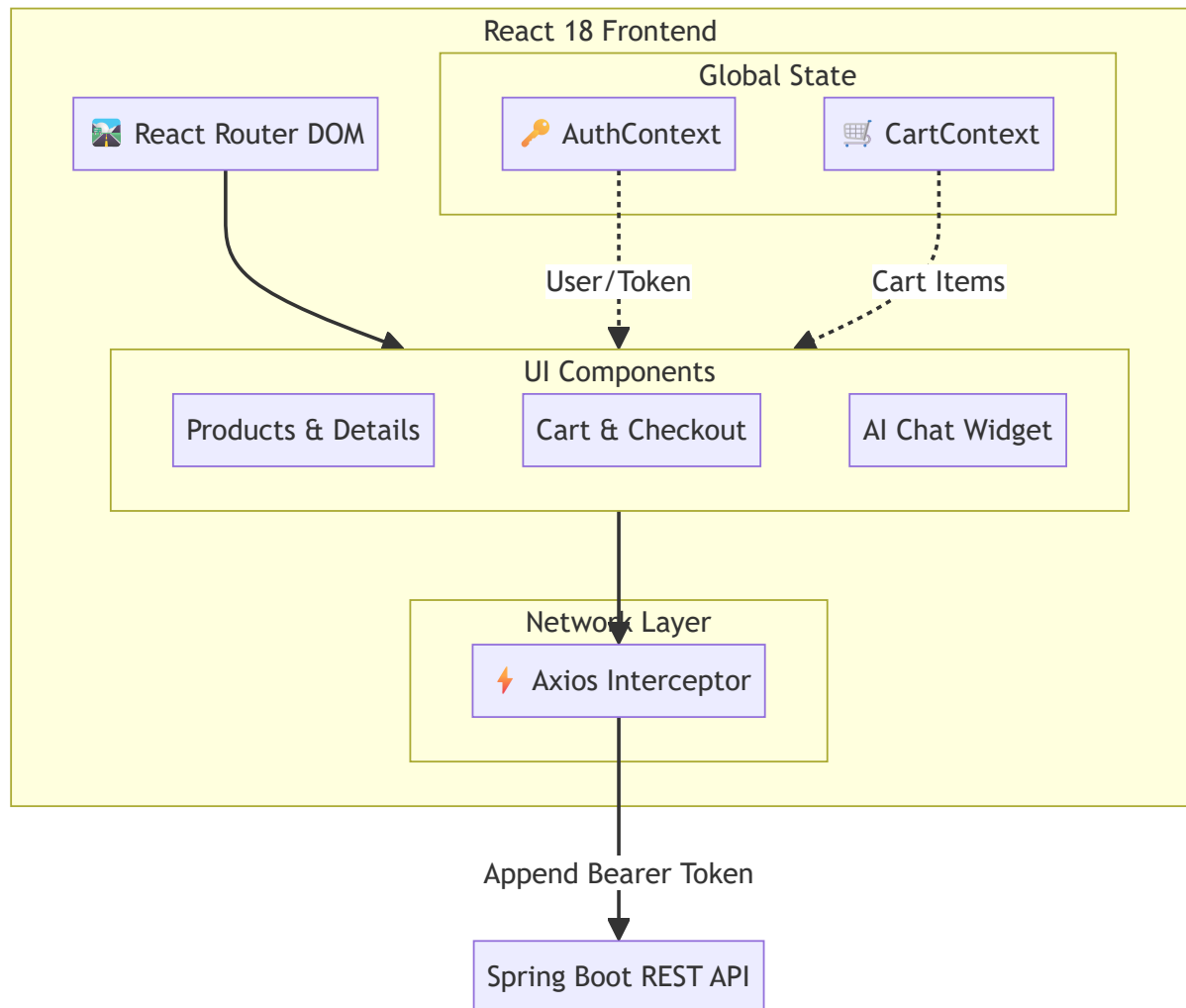
1. End-to-End System Architecture

The system is deployed on AWS with a decoupled frontend and backend. The React SPA is delivered via edge caching, while the Spring Boot backend runs in a containerized Docker environment connecting to a managed relational database and external AI APIs.



2. Frontend Architecture & Workflow

The frontend is a Single Page Application (SPA) built with React 18 and Vite. It relies on global state management and centralized network interceptors to handle data seamlessly.



2.1 State Management (Context API)

AuthContext.jsx: Acts as the central security vault for the browser. On initial load, it checks `localStorage` for a JWT token. If present, it sets the user state.

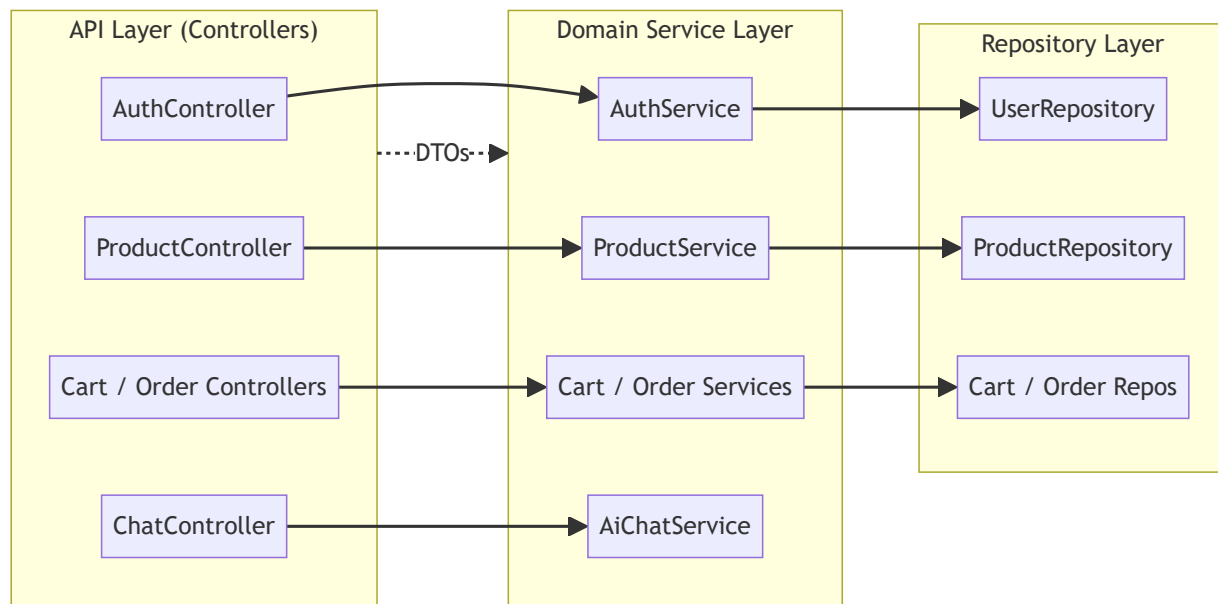
CartContext.jsx: Depends on `AuthContext`. When a user logs in, this context automatically fetches their cart data from the backend. It recalculates totals dynamically.

2.2 Network Layer & Route Protection

- **Axios Interceptor:** Automatically attaches the JWT token to the Authorization: Bearer header for all outgoing API requests.
- **Protected Actions:** If a guest clicks "Add to Cart" or "Write Review", the component checks if (!user). If true, it triggers a warning toast and immediately redirects to /login, preventing unauthorized API calls.

3. Backend Architecture & Workflow

The backend acts as a stateless RESTful API structured as a Microservices-Ready Modular Monolith. Every request must pass through a strict security filter chain before hitting business logic.



3.1 The Request Lifecycle & Security (JWT)

- Incoming Request:** A request arrives at the AWS EC2 server (e.g., POST /api/cart/add).
- JwtAuthenticationFilter:** The Spring Security filter chain intercepts the request and extracts the token.
- Token Validation:** The JwtTokenProvider verifies the cryptographic signature. If valid, it sets the SecurityContextHolder, marking the request as Authenticated.
- DispatcherServlet:** Spring routes the authenticated request to the appropriate controller.

3.2 DTO & Entity Decoupling

Database Entities (like User.java) are never returned directly to the frontend to prevent data leaks. Instead, the Service layer maps Entities into clean ProductDto or OrderDto objects before returning the JSON response.

4. AI Integration Workflow (Google Gemini API)

The platform features an intelligent shopping assistant that guides users based on natural language queries, integrated directly into the Spring Boot backend.

1. **Frontend Prompt:** The user types a query in the React Chat Widget. The frontend sends a JSON payload to `POST /api/ai/chat`.
2. **Backend Orchestration:** The `AiChatService.java` intercepts the query. It constructs a highly specific, hidden system prompt (e.g., *"You are an AI assistant for GK ShopEase..."*) and appends the user's query.
3. **External API Call:** Spring Boot makes a synchronous REST call over HTTPS to the **Google Gemini LLM API**.
4. **Jackson JSON Parsing:** The backend uses Jackson's `ObjectMapper` and `JsonNode` to traverse the complex JSON response tree, extract the exact text response, and discard metadata.
5. **Client Response:** The cleaned string is sent back to the React frontend.

5. Cloud Deployment & CI/CD Workflow

The application relies on dual GitHub Actions pipelines for automated, zero-downtime deployments to Amazon Web Services.

- **Frontend Pipeline (S3 + CloudFront):** On `git push`, GitHub Actions builds the React app via Vite, syncs the assets to an Amazon S3 Bucket using the AWS CLI, and triggers an Amazon CloudFront cache invalidation (`/*`) to deliver the latest UI globally.
- **Backend Pipeline (Docker + EC2):** On `git push`, the runner builds the executable JAR file using Maven. It securely connects to the AWS EC2 instance via SSH/SCP to transfer the JAR. A `systemd` service then rebuilds the Docker container and restarts it, seamlessly reconnecting to the persistent Amazon RDS MySQL database.